ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN



Báo cáo Project 01

Đề tài: Bloxorz solver

Môn học: Cơ sở trí tuệ nhân tạo

Sinh viên thực hiện:
Võ Gia Phúc (24120413)
Phạm Trường Quỳnh (24120426)
Nguyễn Việt Thái (24120441)
Thân Nguyễn Quang Triệu (24120475)
Nguyễn Tấn Vinh (24120492)

Giảng viên hướng dẫn:
Bùi Duy Đăng
Nguyễn Ngọc Đức
Nguyễn Thanh Tình

Ngày 31 tháng 10 năm 2026

Mục lục

Danh mục hình ảnh **iii**

Danh mục bảng **iv**

1 Bảng phân công công việc và tỷ lệ hoàn thành của từng thành viên **1**
1.1 Phân công công việc . . . 1
1.2 Tỷ lệ hoàn thành . . . 2
1.3 Link demo YouTube . . . 2

2 Mô hình hóa bài toán **3**
2.1 Biểu diễn trạng thái và trạng thái ban đầu . . . 3
2.1.1 Vị trí và tư thế của khối . . . 3
2.1.2 Cấu hình đóng/mở của các cầu . . . 5
2.1.3 Trạng thái tách khối và khối đang được điều khiển . . . 6
2.2 Các hành động . . . 6
2.3 Mô hình chuyển trạng thái . . . 7
2.3.1 Các phép lăn khối và chuyển tư thế . . . 7
2.3.2 Kiểm tra biên, ô trống và tính hợp lệ của nước đi . . . 8
2.3.3 Ô dễ vỡ . . . 9
2.3.4 Công tắc mềm, công tắc nặng và cơ chế điều khiển cầu . . . 9
2.3.5 Công tắc tách khối, dịch chuyển và ghép khối . . . 10
2.4 Điều kiện đạt mục tiêu . . . 11
2.5 Định nghĩa và biện luận hàm chi phí . . . 11

3 Các thuật toán tìm kiếm **13**
3.1 Thuật toán Breadth-First Search — BFS . . . 13
3.1.1 Nguyên lý hoạt động . . . 13
3.1.2 Cách áp dụng vào Bloxorz . . . 13
3.1.3 Nhận diện và xử lý trạng thái lặp . . . 14
3.2 Thuật toán Depth-First Search — DFS . . . 15
3.2.1 Nguyên lý hoạt động . . . 15
3.2.2 Cách áp dụng vào Bloxorz . . . 16
3.2.3 Nhận diện trạng thái lặp và phòng tránh chu trình . . . 16

3.3 Thuật toán Uniform-Cost Search — UCS . . . 17
3.3.1 Nguyên lý hoạt động . . . 17
3.3.2 Áp dụng hàm chi phí và tính chi phí tích lũy . . . 17
3.3.3 Cách áp dụng vào Bloxorz . . . 17
3.3.4 Nhận diện và xử lý trạng thái lặp . . . 18
3.4 Thuật toán A* Search . . . 18
3.4.1 Nguyên lý hoạt động và hàm đánh giá ($f(s) = g(s) + h(s)$) . . . 18
3.4.2 Hàm chi phí tích lũy ($g(s)$) . . . 19
3.4.3 Thiết kế và giải thích hàm heuristic ($h(s)$) . . . 20
3.4.4 Lập luận về tính admissible và/hoặc consistent đối với hàm chi phí đã chọn . . . 22
3.4.5 Cách áp dụng vào Bloxorz . . . 23
3.4.6 Nhận diện và xử lý trạng thái lặp . . . 24

4 Thực nghiệm và đánh giá — 20 điểm 25
4.1 Tập kiểm thử cố định dùng chung cho các thuật toán . . . 25
4.2 Mô tả đặc điểm và các nhóm kiểm thử . . . 25
4.3 Phương pháp đo các chỉ số . . . 26
4.3.1 Thời gian tìm kiếm . . . 27
4.3.2 Mức sử dụng bộ nhớ đỉnh . . . 28
4.3.3 Số nút mở rộng . . . 29
4.3.4 Độ dài lời giải — số hành động . . . 29
4.4 Bảng tổng hợp kết quả thực nghiệm . . . 31
4.5 Biểu đồ so sánh các thuật toán . . . 33
4.6 Phân tích kết quả và ưu, nhược điểm của từng thuật toán . . . 36

5 Tài liệu tham khảo 39

A Khai báo sử dụng công cụ AI 40
A.1 Công cụ và phạm vi sử dụng . . . 40
A.2 Prompt tiêu biểu . . . 41
A.3 Kiểm chứng và trách nhiệm . . . 42

Danh mục hình ảnh

1 Thời gian tìm kiếm của các thuật toán, tính bằng giây. 33
2 Mức sử dụng bộ nhớ đỉnh được tracemalloc theo dõi, tính bằng MiB. 33
3 Số lượt mở rộng nút của các thuật toán. Một trạng thái có thể được mở rộng nhiều lần. 34
4 Độ dài lời giải của các thuật toán, tính cả thao tác đổi khối điều khiển (SWITCH_BLOCK). 34
5 Số lần lăn của các thuật toán, chỉ tính UP, DOWN, LEFT và RIGHT, không tính SWITCH_BLOCK. 35

Danh mục bảng

1 Phân công công việc nhóm theo vai trò 1
2 Tỷ lệ hoàn thành công việc của từng thành viên 2
3 Định nghĩa chi phí bước đi theo thứ tự xét điều kiện 12
4 Các thành phần trong hàm đánh giá của A* 19
5 Hai heuristic trong đồ án 20
6 Tóm tắt hai heuristic 23
7 Các nhóm kiểm thử và mục đích đánh giá 26
8 Kết quả thực nghiệm trên các màn 01–05 31
9 Kết quả thực nghiệm trên các màn 06–10 32

1 Bảng phân công công việc và tỷ lệ hoàn thành của từng thành viên

1.1 Phân công công việc

Bảng 1: Phân công công việc nhóm theo vai trò

Vai trò / Người phụ trách	**Trách nhiệm chính**	**Sản phẩm bàn giao**

System Architect & BFS
Phạm Trường Quỳnh (Nhóm trưởng)
	Thiết kế Core Engine (`domain/`, `engine/`).
	Cài đặt thuật toán BFS.
	Viết bộ Unit Tests (`tests/`).
	Quản trị kiến trúc dự án và Git.
		`src/bloxorz/domain/`
		`src/bloxorz/engine/`
		`src/bloxorz/solvers/bfs.py`
		`tests/`

Algorithm Specialist 1 Nguyễn Tấn Vinh	Cài đặt DFS/IDS. Cài đặt UCS với Non-Uniform Cost Function. Biện luận hàm chi phí trong báo cáo.	`src/bloxorz/solvers/dfs.py` `src/bloxorz/solvers/ucs.py`

Algorithm Specialist 2
Võ Gia Phúc
	Cài đặt A* Search.
	Thiết kế các hàm heuristic; chứng minh tính admissible và consistent.
	Cài đặt module đo lường `metrics/`: RAM (`tracemalloc`), thời gian, số node.
		`src/bloxorz/solvers/astar.py`
		`src/bloxorz/solvers/heuristics.py`
		`src/bloxorz/metrics/`

GUI & Animation Lead Thân Nguyễn Quang Triệu	Thiết kế giao diện 3D (bàn chơi và khối sắt). Xây dựng chế độ chơi thủ công (Manual play). Hoạt ảnh tự động lăn khối sắt khi giải (Auto-replay animation). Quay video demo YouTube.	`src/bloxorz/ui/` `src/bloxorz/main.py`

QA, Benchmark & Report
Nguyễn Việt Thái
	Chuẩn bị 10 map test cases JSON có độ khó tăng dần.
	Review mã nguồn các thuật toán tìm kiếm.
	Viết script chạy benchmark tự động cho 4 thuật toán.
	Soạn thảo báo cáo PDF.
		`stages/`
		`src/bloxorz/`
		`experiments/`
		`results/`
		`docs/`

1.2 Tỷ lệ hoàn thành

Bảng 2: Tỷ lệ hoàn thành công việc của từng thành viên

MSSV	**Họ và tên**	**Tỷ lệ hoàn thành (%)**
24120413	Võ Gia Phúc	100
24120426	Phạm Trường Quỳnh	100
24120441	Nguyễn Việt Thái	100
24120475	Thân Nguyễn Quang Triệu	100
24120492	Nguyễn Tấn Vinh	100

1.3 Link demo YouTube

Đường dẫn video demo YouTube: https://youtu.be/A5k8KONmaEg.

2 Mô hình hóa bài toán

Bài toán giải trò chơi Bloxorz được hình thức hóa thành một bài toán tìm kiếm đường đi trong không gian trạng thái thông qua bộ sáu thành phần toán học:

$$\mathcal{P} = \langle S, s_0, A, T, G, c \rangle$$

Trong đó:

- S: Không gian toàn bộ các trạng thái hợp lệ của bàn cờ và khối sắt.
- $s_0 \in S$: Trạng thái ban đầu khi bắt đầu màn chơi.
- A: Tập các hành động có thể xét; tính hợp lệ của từng hành động phụ thuộc vào trạng thái hiện tại.
- $T : S \times A \to S \cup \{\emptyset\}$: Hàm chuyển trạng thái mô tả luật vật lý của trò chơi.
- $G \subseteq S$: Tập các trạng thái mục tiêu (Goal states) thỏa mãn điều kiện chiến thắng.
- $c(s, a, s')$: Hàm chi phí thực hiện hành động a chuyển từ trạng thái s sang trạng thái s' (được chi tiết hóa bởi TV2 tại Mục 2.5).

2.1 Biểu diễn trạng thái và trạng thái ban đầu

Mỗi trạng thái $s \in S$ biểu diễn cấu hình hoàn chỉnh của bàn cờ và khối sắt tại một thời điểm, được hình thức hóa thông qua một bộ 4 thành phần:

$$s = \langle (r_1, c_1), (r_2, c_2), a_{\text{cube}}, B \rangle$$

2.1.1 Vị trí và tư thế của khối

Vị trí của khối sắt trên mặt lưới được xác định thông qua tọa độ hàng và cột (0-based index) của hai khối lập phương đơn vị $1 \times 1 \times 1$ cấu thành nên nó: (r_1, c_1) và (r_2, c_2).

- **Quy chuẩn tọa độ chuẩn hóa (Canonical Representation):** Do hai khối lập phương đơn vị có tính đối xứng vật lý, để tránh việc trùng lặp trạng thái trong bộ nhớ băm (ví dụ: việc hoán vị giữa (r_1, c_1) và (r_2, c_2) vẫn biểu diễn cùng một khối sắt trên bàn cờ), hệ thống áp dụng quy tắc sắp xếp từ điển bắt buộc:

$$(r_1, c_1) \leq (r_2, c_2) \iff (r_1 < r_2) \vee (r_1 = r_2 \wedge c_1 \leq c_2)$$

  Khi cần hoán đổi hai tọa độ của khối đang tách, hệ thống đồng thời đổi $a_{\text{cube}} \leftarrow 1 - a_{\text{cube}}$ để giữ đúng khối đang được điều khiển. Việc chuẩn hóa được thực hiện trong hàm chuyển trạng thái; lớp `State` không tự sắp xếp tọa độ khi khởi tạo.

- **Ba tư thế hình học của khối sắt liền khối ($a_{\text{cube}} = -1$):** Khi hai khối đơn vị liên kết liền thành một khối hộp chữ nhật $1 \times 1 \times 2$, khối sắt có đúng 3 tư thế hình học trong không gian 3 chiều:

  1. **Đứng thẳng (STANDING):**

$$(r_1, c_1) = (r_2, c_2)$$

     Khối sắt đứng vuông góc với mặt phẳng bàn cờ, mặt tiếp xúc đáy là một hình vuông đơn vị 1×1. Khối chiếm đúng 1 ô duy nhất trên lưới bàn cờ.

  2. **Nằm ngang theo hàng (LYING_ROW):**

$$r_1 = r_2 \quad \text{và} \quad c_2 = c_1 + 1$$

     Khối sắt nằm song song với các trục hàng, mặt tiếp xúc là hình chữ nhật 1×2. Khối chiếm đúng 2 ô kề sát nhau trên cùng một hàng.

  3. **Nằm dọc theo cột (LYING_COL):**

$$c_1 = c_2 \quad \text{và} \quad r_2 = r_1 + 1$$

Khối sắt nằm song song với các trục cột, mặt tiếp xúc là hình chữ nhật 2×1. Khối chiếm đúng 2 ô kề sát nhau trên cùng một cột.

- **Trạng thái ban đầu (s_0):** Khối sắt luôn bắt đầu ở tư thế đứng thẳng (STANDING) ngay tại vị trí xuất phát ký hiệu `'S'` trên bản đồ:

$$(r_1, c_1) = (r_2, c_2) = (r_{\text{start}}, c_{\text{start}}), \quad a_{\text{cube}} = -1, \quad B = B_0$$

2.1.2 Cấu hình đóng/mở của các cầu

Trên bàn cờ có thể xuất hiện k nhóm cầu nhằm kết nối các vùng sàn rời rạc với nhau ($k \geq 0$). Mỗi nhóm có thể gồm nhiều ô cầu và dùng chung một trạng thái đóng/mở.

- Trạng thái đóng/mở của toàn bộ hệ thống cầu được biểu diễn hình thức bằng một bộ tuple bất biến (immutable tuple) gồm k phần tử nhị phân:

$$B = \langle b_0, b_1, \ldots, b_{k-1} \rangle \in \{0, 1\}^k$$

- **Ý nghĩa giá trị:**
  - $b_i = 1$: Cây cầu thứ i đang ở trạng thái **mở** (active). Mặt sàn tại ô cầu có bề mặt vững chắc, khối sắt có thể lăn qua an toàn.
  - $b_i = 0$: Cây cầu thứ i đang ở trạng thái **đóng** (inactive). Vị trí này hoàn toàn rỗng và được hệ thống xem như khoảng không vô định (void). Nếu khối sắt di chuyển vào vị trí này, khối sẽ rơi xuống vực.
- **Cấu hình ban đầu B_0:** Trạng thái ban đầu của từng cây cầu được nạp từ cấu hình màn chơi JSON: các ô cầu ký hiệu chữ hoa `'B'` có giá trị khởi tạo là 1 (mở), các ô cầu ký hiệu chữ thường `'b'` có giá trị khởi tạo là 0 (đóng).

2.1.3 Trạng thái tách khối và khối đang được điều khiển

Trò chơi sở hữu cơ chế nâng cao cho phép khối sắt tách thành hai khối lập phương đơn vị độc lập. Biến chỉ số $a_{\text{cube}} \in \{-1, 0, 1\}$ quản lý chặt chẽ cơ chế phân tách và quyền nhận điều khiển này:

- $a_{\text{cube}} = -1$: Khối sắt đang ở trạng thái **hợp nhất** (Unified Block $1 \times 1 \times 2$). Mọi thao tác di chuyển áp dụng đồng thời lên toàn bộ khối sắt.
- $a_{\text{cube}} = 0$: Khối sắt đang ở trạng thái **phân tách** (Split State), và **Khối 0** (tọa lạc tại tọa độ (r_1, c_1)) đang nhận quyền điều khiển trực tiếp từ người chơi hoặc thuật toán tìm kiếm. Khối 1 tại (r_2, c_2) giữ nguyên vị trí đứng yên.
- $a_{\text{cube}} = 1$: Khối sắt đang ở trạng thái **phân tách** (Split State), và **Khối 1** (tọa lạc tại tọa độ (r_2, c_2)) đang nhận quyền điều khiển trực tiếp. Khối 0 tại (r_1, c_1) đứng yên.

2.2 Các hành động

Tập hành động có thể xét của trò chơi gồm 5 phần tử; không phải hành động nào cũng hợp lệ ở mọi trạng thái:

$$A = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}, \text{SWITCH_BLOCK}\}$$

1. **Nhóm hành động lăn không gian ($\{\mathbf{UP}, \mathbf{DOWN}, \mathbf{LEFT}, \mathbf{RIGHT}\}$):** Thực hiện phép lăn lật khối trên mặt phẳng lưới theo 4 hướng la bàn (Lên trên, Xuống dưới, Sang trái, Sang phải). Khi khối ở trạng thái liền ($a_{\text{cube}} = -1$), hành động sẽ lăn toàn bộ khối sắt làm thay đổi đồng thời vị trí và tư thế hình học. Khi khối ở trạng thái phân tách ($a_{\text{cube}} \geq 0$), hành động sẽ lăn dịch chuyển duy nhất khối lập phương nhỏ đang nhận điều khiển đi 1 ô theo hướng tương ứng.
2. **Hành động chuyển đổi quyền điều khiển (SWITCH_BLOCK):** Thực hiện

chuyển quyền điều khiển qua lại giữa hai khối lập phương đơn vị nhỏ:

$$a_{\text{cube}} \leftarrow 1 - a_{\text{cube}}$$

Quy tắc áp dụng: Hành động SWITCH_BLOCK **chỉ hợp lệ** khi hệ thống đang ở trạng thái phân tách ($a_{\text{cube}} \geq 0$). Hành động này hoàn toàn không làm thay đổi tọa độ không gian của các khối cũng như cấu hình đóng/mở của các cây cầu.

2.3 Mô hình chuyển trạng thái

Hàm chuyển trạng thái $T : S \times A \to S \cup \{\emptyset\}$ mô tả sự chuyển biến từ trạng thái hiện tại s dưới tác động của hành động a sang trạng thái mới $s' = T(s, a)$. Nếu hành động dẫn đến một cấu hình vi phạm luật vật lý hoặc khiến khối rơi khỏi bàn cờ, hàm sẽ trả về giá trị rỗng $\emptyset$ (bước đi không hợp lệ).

2.3.1 Các phép lăn khối và chuyển tư thế

Động học lật khối sắt tuân thủ chính xác phép biến đổi quay không gian 90° quanh cạnh đáy tiếp xúc với mặt sàn:

- **Trường hợp 1: Khối đang ở tư thế STANDING tại ô** (r, c)**:**
  - Lăn UP: Khối chuyển sang tư thế LYING_COL, chiếm 2 ô $(r-2, c)$ và $(r-1, c)$.
  - Lăn DOWN: Khối chuyển sang tư thế LYING_COL, chiếm 2 ô $(r+1, c)$ và $(r+2, c)$.
  - Lăn LEFT: Khối chuyển sang tư thế LYING_ROW, chiếm 2 ô $(r, c-2)$ và $(r, c-1)$.
  - Lăn RIGHT: Khối chuyển sang tư thế LYING_ROW, chiếm 2 ô $(r, c+1)$ và $(r, c+2)$.
- **Trường hợp 2: Khối đang ở tư thế LYING_ROW chiếm 2 ô** (r, c) **và** $(r, c+1)$**:**
  - Lăn UP: Khối giữ nguyên tư thế LYING_ROW, tịnh tiến hàng: chiếm 2 ô $(r-1, c)$ và $(r-1, c+1)$.

- Lăn DOWN: Khối giữ nguyên tư thế LYING_ROW, tịnh tiến hàng: chiếm 2 ô $(r+1, c)$ và $(r+1, c+1)$.
- Lăn LEFT: Khối dựng đứng thành STANDING tại ô $(r, c-1)$.
- Lăn RIGHT: Khối dựng đứng thành STANDING tại ô $(r, c+2)$.

- **Trường hợp 3: Khối đang ở tư thế LYING_COL chiếm 2 ô** (r, c) **và** $(r+1, c)$**:**
  - Lăn UP: Khối dựng đứng thành STANDING tại ô $(r-1, c)$.
  - Lăn DOWN: Khối dựng đứng thành STANDING tại ô $(r+2, c)$.
  - Lăn LEFT: Khối giữ nguyên tư thế LYING_COL, tịnh tiến cột: chiếm 2 ô $(r, c-1)$ và $(r+1, c-1)$.
  - Lăn RIGHT: Khối giữ nguyên tư thế LYING_COL, tịnh tiến cột: chiếm 2 ô $(r, c+1)$ và $(r+1, c+1)$.

- **Trường hợp 4: Khối đang ở trạng thái phân tách (**$a_{\mathbf{cube}} \geq 0$**):**
  - Khối lập phương nhỏ $1 \times 1 \times 1$ đang nhận quyền điều khiển dịch chuyển đúng 1 ô đơn vị theo hướng lăn tương ứng:

$$\text{UP} : (r-1, c); \quad \text{DOWN} : (r+1, c); \quad \text{LEFT} : (r, c-1); \quad \text{RIGHT} : (r, c+1)$$

  - Khối nhỏ còn lại hoàn toàn giữ nguyên tọa độ.

2.3.2 Kiểm tra biên, ô trống và tính hợp lệ của nước đi

Sau khi tính toán các tọa độ mới cells(s'), hệ thống kiểm tra các điều kiện an toàn nghiêm ngặt:

1. **Kiểm tra biên bàn cờ:** Toàn bộ các ô mà khối chiếm giữ phải nằm trong kích thước lưới bản đồ $R \times C$:

$$\forall (r, c) \in \text{cells}(s') : 0 \leq r < R \quad \text{và} \quad 0 \leq c < C$$

2. **Kiểm tra ô khoảng không (Void):** Không một phần nào của khối được đặt lên ô trống vô định:
$$\forall (r, c) \in \text{cells}(s') : \text{grid}[r][c] \neq \text{' '}$$

3. **Kiểm tra cầu đang đóng:** Nếu một ô (r, c) là vị trí cầu thứ i, ô đó chỉ hợp lệ khi và chỉ khi cây cầu đang ở trạng thái mở ($b_i = 1$). Nếu cầu đang đóng ($b_i = 0$), ô đó được đối xử như ô khoảng không.

Nếu vi phạm bất kỳ điều kiện nào trong ba điều kiện trên, khối sắt rơi khỏi bàn cờ và nước đi bị coi là không hợp lệ: $T(s, a) = \emptyset$.
Việc kiểm tra được thực hiện trước khi xử lý công tắc và được lặp lại sau khi cập nhật cầu, dịch chuyển và ghép khối. Nếu một công tắc làm đóng cầu đang đỡ khối ở trạng thái cuối, nước đi cũng bị loại bỏ.

2.3.3 Ô dễ vỡ

Các ô sàn cam mỏng manh được định nghĩa bằng ký hiệu `'-'` (Fragile Tiles).

- **Quy tắc với khối liền:**
  - Khi khối ở tư thế nằm (LYING_ROW hoặc LYING_COL), khối được phép nằm trên ô dễ vỡ.
  - Khi khối ở tư thế đứng thẳng (STANDING) trên ô dễ vỡ, nước đi không hợp lệ: $T(s, a) = \emptyset$.
- **Quy tắc khối phân tách:** Khối lập phương nhỏ $1 \times 1 \times 1$ **được phép đứng và đi qua ô dễ vỡ an toàn**. Mã nguồn kiểm tra trực tiếp tư thế và trạng thái tách, không mô phỏng khối lượng hay áp suất.

2.3.4 Công tắc mềm, công tắc nặng và cơ chế điều khiển cầu

Khi khối sắt di chuyển vào các ô công tắc, hệ thống kích hoạt cơ chế cập nhật trạng thái cầu:

- **Công tắc mềm (`'O'` — Soft Switch):** Được kích hoạt khi khối đứng lên, khi khối nằm chiếm thêm ô công tắc mới so với vị trí trước đó, hoặc khi khối con đang di chuyển bước lên ô công tắc. Khối con đứng yên không kích hoạt lại công tắc; hành động `SWITCH_BLOCK` cũng không kích hoạt công tắc. Sau dịch chuyển tức thời, công tắc mềm tại cả hai vị trí đến đều được xử lý.

- **Công tắc nặng (`'X'` — Heavy Switch):** Đòi hỏi áp lực lớn. **Chỉ kích hoạt duy nhất khi khối sắt** $1 \times 1 \times 2$ **đứng thẳng (STANDING)** trên ô `'X'`. Mọi trường hợp khối nằm đè qua hoặc khối nhỏ $1 \times 1 \times 1$ dẫm lên đều không đủ trọng lượng để kích hoạt.

- **Ba cơ chế điều khiển trạng thái cầu liên kết:** Mỗi công tắc có thể liên kết với một hoặc nhiều cây cầu với các chế độ điều khiển sau:

  – Chế độ bật (`on`): Cầu được mở ra vững chắc: $b_i \leftarrow 1$.

  – Chế độ tắt (`off`): Cầu bị thu lại thành khoảng không: $b_i \leftarrow 0$.

  – Chế độ đảo (`swap`): Đảo ngược trạng thái hiện tại của cầu: $b_i \leftarrow 1 - b_i$.

2.3.5 Công tắc tách khối, dịch chuyển và ghép khối

- **Kích hoạt công tắc phân tách (`'T'` — Split Switch):** Công tắc phân tách chỉ được kích hoạt khi khối sắt $1 \times 1 \times 2$ **đứng thẳng (STANDING)** lên ô `'T'`.

- **Cơ chế dịch chuyển tức thời (Teleportation):** Khi kích hoạt, khối sắt lập tức tách đôi thành hai khối lập phương $1 \times 1 \times 1$ và dịch chuyển đến hai tọa độ định trước trong file cấu hình bản đồ. Khối con được điều khiển ban đầu do cấu hình lựa chọn, không luôn là khối 0. Nếu cần đổi thứ tự tọa độ để chuẩn hóa, chỉ số điều khiển cũng được đổi tương ứng. Các tác động lên cầu gắn với ô công tắc nguồn được xử lý trước khi kiểm tra hai vị trí đến; sau đó hệ thống xử lý công tắc mềm tại các vị trí này.

- **Cơ chế tự động ghép khối (Auto-merge):** Sau khi di chuyển khối con hoặc dịch chuyển tức thời và xử lý công tắc, hệ thống tự động kiểm tra khoảng cách Manhattan

giữa hai khối nhỏ:

$$d_{\text{Manhattan}} = |r_1 - r_2| + |c_1 - c_2|$$

Nếu $d_{\text{Manhattan}} = 1$ (hai khối lập phương đơn vị nằm ở hai ô kề sát nhau theo cạnh), hai khối sẽ **tự động hút và sáp nhập tức thời** thành một khối sắt $1 \times 1 \times 2$ thống nhất:

- Nếu $r_1 = r_2$: Sáp nhập thành tư thế LYING_ROW.
- Nếu $c_1 = c_2$: Sáp nhập thành tư thế LYING_COL.
- Quyền điều khiển chuyển về trạng thái hợp nhất: $a_{\text{cube}} \leftarrow -1$.

2.4 Điều kiện đạt mục tiêu

Hàm kiểm tra mục tiêu GoalTest : $S \to \{\text{True}, \text{False}\}$ xác định trạng thái có thuộc tập đích G hay không. Trạng thái s được công nhận là trạng thái đích khi và chỉ khi:

$$\begin{aligned}\text{GoalTest}(s) = \text{True} &\iff s \in G \\ &\iff (a_{\text{cube}} = -1) \wedge (r_1 = r_2 = r_{\text{goal}}) \\ &\quad \wedge (c_1 = c_2 = c_{\text{goal}})\end{aligned}$$

1. Khối sắt phải ở trạng thái **hợp nhất liền khối** ($a_{\text{cube}} = -1$).
2. Khối sắt phải ở tư thế **đứng thẳng (STANDING)** ngay tại tọa độ ô lỗ đích ký hiệu `'H'`: $(r_1, c_1) = (r_2, c_2) = (r_{\text{goal}}, c_{\text{goal}})$.

Ràng buộc loại trừ: Hệ thống kiên quyết từ chối công nhận chiến thắng nếu khối sắt nằm đè qua ô `'H'`, hoặc nếu chỉ có một khối lập phương nhỏ $1 \times 1 \times 1$ đứng tại ô `'H'` khi chưa thực hiện ghép khối hoàn chỉnh.

2.5 Định nghĩa và biện luận hàm chi phí

UCS và A* dùng chung hàm chi phí bước đi $c(s, a, s')$ trong `src/bloxorz/solvers/costs.py`. Các điều kiện được xét theo thứ tự từ trên xuống; điều kiện đầu tiên thỏa mãn quyết

định chi phí của hành động.

Bảng 3: Định nghĩa chi phí bước đi theo thứ tự xét điều kiện

Điều kiện	**Chi phí**
Hành động `SWITCH_BLOCK`, đổi khối con đang điều khiển	1
Cấu hình cầu thay đổi, hoặc khối chuyển từ chưa tách sang đã tách	1
Khối liền chiếm ít nhất một ô dễ vỡ ở trạng thái kế tiếp	4
Các trường hợp còn lại	2

Nhóm chọn 2 làm chi phí cơ sở để có thể đặt một số hành động ở mức thấp hơn mà vẫn giữ mọi chi phí là số nguyên dương. Mức 4 dành cho việc đưa khối liền lên ô dễ vỡ, nhằm hạn chế lựa chọn này nếu có đường đi khác. Đây là mức phạt do nhóm đặt ra; tính hợp lệ của nước đi vẫn được kiểm tra riêng theo luật chơi.

Mức 1 dành cho thao tác đổi khối điều khiển, thay đổi cấu hình cầu hoặc bắt đầu tách khối. Với cầu, quy tắc áp dụng cả khi mở, đóng hoặc đảo trạng thái, không chỉ khi thao tác đó mở thêm lối đi. Nếu đạp công tắc nhưng cấu hình cầu không đổi và khối không vừa tách, chi phí được xét tiếp theo hai dòng còn lại của bảng. Vì vậy, trường hợp này vẫn có thể nhận chi phí 4 nếu khối liền nằm trên ô dễ vỡ.

Thứ tự xét điều kiện cũng có ý nghĩa: nếu một hành động vừa làm thay đổi cấu hình cầu vừa đưa khối liền lên ô dễ vỡ, chi phí là 1. Các khối con đang tách không chịu mức phạt 4. Khối liền đứng trên ô dễ vỡ là trạng thái không hợp lệ, nên không tạo thành một bước đi trong đồ thị tìm kiếm.

Với lời giải $\pi = (a_1, \ldots, a_k)$, tổng chi phí được tính bằng:

$$C(\pi) = \sum_{i=1}^{k} c(s_{i-1}, a_i, s_i).$$

Do chi phí các hành động không đồng nhất, lời giải có ít hành động nhất chưa chắc có tổng chi phí nhỏ nhất. Mọi bước hợp lệ đều có chi phí ít nhất $c_{\min} = 1$; đây là cơ sở để UCS bảo đảm tối ưu và để xây dựng cận dưới trong heuristic của A*.

3 Các thuật toán tìm kiếm

3.1 Thuật toán Breadth-First Search — BFS

3.1.1 Nguyên lý hoạt động

Thuật toán tìm kiếm theo chiều rộng (Breadth-First Search — BFS) là thuật toán tìm kiếm không có thông tin (uninformed search) khám phá không gian trạng thái theo từng tầng độ sâu (level-by-level).

- **Cơ chế tập biên (Frontier):** Thuật toán sử dụng cấu trúc hàng đợi FIFO (First-In, First-Out) để quản lý tập các nút chờ mở rộng. Tất cả các nút ở độ sâu d được đảm bảo duyệt xong hoàn toàn trước khi thuật toán mở rộng bất kỳ nút nào ở độ sâu $d+1$.
- **Đặc tính lý thuyết:**
  - **Tính đầy đủ (Completeness):** Do không gian trạng thái Bloxorz là hữu hạn ($|S| < \infty$) và hệ số phân nhánh bị chặn trên ($b \leq 5$), BFS luôn đảm bảo tìm thấy đường đi tới đích nếu tồn tại lời giải.
  - **Tính tối ưu bước đi (Step-optimal):** Khi mỗi hành động được tính là một bước, kể cả `SWITCH_BLOCK`, đường đi đầu tiên BFS tìm thấy tới đích có tổng số hành động (Path Length) nhỏ nhất. BFS không tối ưu theo hàm chi phí không đồng nhất dùng cho UCS và A*.
  - **Độ phức tạp:** Độ phức tạp thời gian và không gian đều là $\mathcal{O}(b^d)$, trong đó b là hệ số nhánh hiệu dụng và d là độ sâu của nghiệm tối ưu.

3.1.2 Cách áp dụng vào Bloxorz

- **Khởi tạo:**
  - Tập biên Frontier được cài đặt bằng hàng đợi hai đầu `collections.deque` trong Python để đảm bảo các thao tác lấy phần tử `popleft()` và chèn phần tử `append()` đạt hiệu năng $\mathcal{O}(1)$.

- Mỗi phần tử trong hàng đợi là một đối tượng trạng thái `State`. Thuật toán lưu riêng `parent_map`, ánh xạ mỗi trạng thái con đến cặp `(trạng thái cha, hành động)` để dựng lại đường đi sau khi gặp đích.
- Kiểm tra trạng thái ban đầu trước vòng lặp. Nếu đã là đích, trả về chuỗi hành động rỗng và số nút mở rộng bằng 0. Nếu chưa phải đích, đưa `initial_state` vào Frontier và thêm vào tập trạng thái đã phát hiện `visited`.

- **Vòng lặp tìm kiếm:**

1. Khi Frontier không rỗng, lấy trạng thái đầu tiên `current_state` ra khỏi hàng đợi.
2. Tăng bộ đếm số nút mở rộng một đơn vị.
3. Lần lượt thử các hành động theo thứ tự `UP`, `DOWN`, `LEFT`, `RIGHT`, `SWITCH_BLOCK`; bỏ qua `SWITCH_BLOCK` khi khối chưa tách.
4. Với mỗi hành động a, gọi hàm chuyển trạng thái của engine:

$$s_{\text{next}} = T(\text{current_state}, a)$$

5. Nếu $s_{\text{next}} \neq \emptyset$ (bước đi hợp lệ) và $s_{\text{next}} \notin$ visited:
   - Thêm s_{next} vào tập `visited`.
   - Lưu `parent_map[s_next] = (current_state, a)`.
   - Kiểm tra GoalTest(s_{next}). Nếu đúng, dừng tìm kiếm, lần ngược `parent_map` từ đích đến xuất phát rồi đảo chuỗi hành động để trả về lời giải.
   - Nếu chưa phải đích, đưa `s_next` vào cuối hàng đợi Frontier.
6. Nếu hàng đợi rỗng mà không tìm thấy trạng thái đích, thuật toán trả về kết quả thất bại (vô nghiệm).

3.1.3 Nhận diện và xử lý trạng thái lặp

- **Thách thức của bài toán:** Không gian trạng thái của Bloxorz có các chu trình, chẳng hạn lăn sang phải rồi lăn ngược lại để trở về trạng thái ban đầu. Nếu không có

cơ chế quản lý trạng thái lặp, thuật toán có thể tiếp tục sinh lại cùng các trạng thái, làm tăng thời gian và bộ nhớ hoặc không kết thúc khi không có lời giải.

- **Thiết kế bất biến của đối tượng State:** Để tối ưu hóa tốc độ kiểm tra lặp trong bảng băm, lớp `State` trong kiến trúc mã nguồn được thiết kế dưới dạng `@dataclass(frozen=True)` với các đặc tính:
  - Tọa độ canonical của các trạng thái do engine sinh ra được sắp xếp theo thứ tự từ điển.
  - Cấu hình cầu B được lưu trữ bằng `tuple` các giá trị Boolean thay vì `list` có thể biến đổi.
  - Nhờ đó, trạng thái có thể băm để lưu trong `visited`. Chi phí băm và so sánh còn phụ thuộc vào số nhóm cầu k; khi coi kích thước trạng thái của một màn là cố định, thao tác tra cứu tập có thời gian trung bình $\mathcal{O}(1)$.
- **Cơ chế cắt tỉa nhánh (Pruning) bằng Graph-search:** Thuật toán duy trì tập băm `visited`, ban đầu chứa trạng thái xuất phát. Kỹ thuật **Early Visited Insertion** (thêm trạng thái kế tiếp vào `visited` ngay tại thời điểm nó được sinh ra thay vì đợi đến lúc lấy ra khỏi hàng đợi) được áp dụng. Điều này ngăn việc đưa cùng một trạng thái vào hàng đợi nhiều lần và tránh lãng phí thời gian, bộ nhớ do các lần xét trùng.

3.2 Thuật toán Depth-First Search — DFS

3.2.1 Nguyên lý hoạt động

DFS dùng ngăn xếp để ưu tiên xét trạng thái vừa được đưa vào gần nhất. Thuật toán tiếp tục tìm kiếm từ trạng thái đó; khi không còn trạng thái mới trên nhánh đang xét, nó lấy trạng thái chờ tiếp theo trong ngăn xếp. Nhóm cài đặt bằng vòng lặp và ngăn xếp tường minh, không dùng đệ quy hoặc IDS.

DFS dừng khi tìm được lời giải đầu tiên. Vì không so sánh độ dài hay tổng chi phí giữa các lời giải, thuật toán không bảo đảm tối ưu theo hai tiêu chí này.

3.2.2 Cách áp dụng vào Bloxorz

DFS sử dụng mô hình trạng thái, hành động và quy tắc chuyển trạng thái đã trình bày ở chương 2. Bộ giải khởi tạo từ vị trí xuất phát và cấu hình cầu ban đầu, sau đó đưa trạng thái này vào ngăn xếp. Nếu trạng thái ban đầu đã là đích, thuật toán trả về chuỗi hành động rỗng và số nút mở rộng bằng 0.

Trong vòng lặp, DFS lấy một trạng thái khỏi ngăn xếp, tăng bộ đếm nút mở rộng rồi xét hành động theo thứ tự `SWITCH_BLOCK`, `RIGHT`, `LEFT`, `DOWN`, `UP`. Hành động đổi khối được bỏ qua khi khối chưa tách. Cách đưa các trạng thái con vào ngăn xếp theo thứ tự này giúp ưu tiên lấy trạng thái sinh bởi `UP` ra xét tiếp, nếu trạng thái đó hợp lệ, chưa được thăm và thuật toán chưa gặp đích. Cần phân biệt thứ tự sinh trạng thái con với thứ tự lấy chúng khỏi ngăn xếp.

Mỗi trạng thái con hợp lệ và chưa được thăm được ghi nhận cùng trạng thái cha và hành động đã dùng. **Điều kiện đích được kiểm tra ngay khi sinh trạng thái con**, trước khi đưa nó vào ngăn xếp. Khi gặp đích, DFS dừng và lần ngược các trạng thái cha để dựng lại lời giải. Nếu ngăn xếp rỗng mà chưa gặp đích, thuật toán báo không tìm thấy lời giải.

3.2.3 Nhận diện trạng thái lặp và phòng tránh chu trình

Tập `visited` lưu các trạng thái đã được phát hiện. Trạng thái được đánh dấu ngay khi sinh, nên mỗi trạng thái được đưa vào ngăn xếp tối đa một lần. Cách này ngăn việc đi lặp trong chu trình và giúp DFS tìm được lời giải nếu có trên không gian trạng thái hữu hạn của Bloxorz.

Việc so sánh trạng thái xét cả vị trí khối, cấu hình cầu và khối con đang được điều khiển. Hai trạng thái có cùng vị trí nhưng khác cấu hình cầu vẫn được xem là khác nhau. Bộ giải dùng trạng thái bất biến, có thể băm để lưu trong tập; thao tác tra cứu tập có thời gian trung bình hằng số khi xét kích thước biểu diễn trạng thái cố định.

DFS còn lưu `parent_map` để dựng lại đường đi. Vì giữ cả tập đã thăm và thông tin trạng thái cha, cài đặt này không chỉ lưu riêng nhánh đang xét; không thể áp dụng trực tiếp nhận xét về bộ nhớ của DFS dạng cây cho bộ giải hiện tại.

3.3 Thuật toán Uniform-Cost Search — UCS

3.3.1 Nguyên lý hoạt động

UCS dùng hàng đợi ưu tiên để chọn trạng thái có chi phí tích lũy $g(s)$ nhỏ nhất. Thuật toán xét đường đi theo chi phí thay vì số hành động. Khi mọi hành động có cùng chi phí, UCS tìm lời giải ít hành động nhất như BFS, dù thời điểm kiểm tra đích và cách cài đặt có thể làm số nút mở rộng khác nhau.

3.3.2 Áp dụng hàm chi phí và tính chi phí tích lũy

Trạng thái xuất phát có $g(s_0) = 0$. Khi xét hành động a từ s đến s', thuật toán tính chi phí của đường đi mới:

$$g_{\text{mới}} = g(s) + c(s, a, s').$$

Hàm c được xác định ở mục 2.5. Nếu $g_{\text{mới}}$ nhỏ hơn chi phí tốt nhất đã lưu cho s', UCS cập nhật `best_g`, lưu trạng thái cha và đưa phần tử mới vào hàng đợi ưu tiên. Khi hai đường có cùng chi phí, bộ giải giữ đường đã ghi nhận trước đó.
Mỗi phần tử của hàng đợi có dạng $(g, \text{số thứ tự}, s)$. Số thứ tự tăng dần giúp ưu tiên phần tử được đưa vào trước khi hai giá trị g bằng nhau, đồng thời tránh phải so sánh trực tiếp các đối tượng trạng thái.

3.3.3 Cách áp dụng vào Bloxorz

UCS sử dụng cùng mô hình bài toán ở chương 2. Sau khi khởi tạo trạng thái xuất phát, thuật toán lần lượt lấy phần tử có g nhỏ nhất khỏi hàng đợi. Nếu chi phí của phần tử lớn hơn `best_g` hiện tại, phần tử đó bị bỏ qua vì đã có đường đi tốt hơn đến cùng trạng thái.
Điều kiện đích được kiểm tra sau bước loại bỏ phần tử cũ và trước khi mở rộng nút. Không dừng ngay khi sinh đích, vì lúc đó có thể vẫn còn một đường đi rẻ hơn chưa được xét. Nếu gặp đích, UCS dựng lại chuỗi hành động từ `parent_map`; nếu chưa gặp, thuật toán tăng bộ đếm và thử các hành động theo thứ tự `UP`, `DOWN`, `LEFT`, `RIGHT`, `SWITCH_BLOCK`.

Hành động đổi khối chỉ được xét khi khối đang tách.

Với chi phí bước đi dương, các phần tử hợp lệ được lấy ra theo thứ tự g không giảm. Do đó, khi đích được lấy ra lần đầu, chi phí của nó đã nhỏ nhất. UCS bảo đảm tìm được lời giải tối ưu về chi phí trên không gian trạng thái hữu hạn; nếu không có lời giải, hàng đợi cuối cùng sẽ rỗng. Trạng thái đích không được tính là một nút mở rộng, kể cả khi trạng thái ban đầu đã là đích.

3.3.4 Nhận diện và xử lý trạng thái lặp

UCS dùng `best_g` thay cho việc đánh dấu đã thăm ngay khi sinh nút. Một trạng thái có thể được đưa vào hàng đợi nhiều lần trước khi được mở rộng, nếu tìm thấy các đường đi có chi phí giảm dần. Các phần tử cũ được giữ trong hàng đợi và bị bỏ qua khi lấy ra nếu chi phí của chúng lớn hơn `best_g`.

Tuy nhiên, **đưa vào hàng đợi nhiều lần không đồng nghĩa với mở rộng nhiều lần**. Với UCS hiện tại và chi phí bước đi dương, khi một trạng thái được lấy ra bằng phần tử hợp lệ đầu tiên, chi phí của nó đã tối ưu. Vì vậy, mỗi trạng thái được mở rộng tối đa một lần. Điều này khác với A* dùng heuristic chưa được bảo đảm consistent, nơi việc mở rộng lại trạng thái có thể cần thiết.

3.4 Thuật toán A* Search

A* dùng thêm thông tin ước lượng về phần đường còn lại để chọn trạng thái cần xét tiếp. Trong đồ án, A* dùng cùng hàm chi phí với UCS ở mục 2.5; điểm khác biệt nằm ở cách sắp xếp thứ tự tìm kiếm. Mục tiêu của A* là tìm lời giải có tổng chi phí nhỏ nhất, không nhất thiết có ít hành động nhất.

3.4.1 Nguyên lý hoạt động và hàm đánh giá ($f(s) = g(s) + h(s)$)

Với mỗi trạng thái s, A* sử dụng:

Bảng 4: Các thành phần trong hàm đánh giá của A*

Ký hiệu	**Ý nghĩa**
$g(s)$	Chi phí tích lũy trên đường đi từ trạng thái xuất phát đến s
$h(s)$	Chi phí còn lại được ước lượng từ s đến đích
$f(s) = g(s) + h(s)$	Giá trị dùng để ưu tiên trạng thái cần xét tiếp

Các trạng thái chờ xét được lưu trong hàng đợi ưu tiên, gọi là OPEN. Mỗi lần lặp, thuật toán lấy phần tử có f nhỏ nhất. Nếu chi phí g của phần tử này lớn hơn chi phí tốt nhất đã lưu cho cùng trạng thái, phần tử bị bỏ qua.

Nếu trạng thái vừa lấy ra là đích, thuật toán dừng và dựng lại lời giải. Nếu chưa phải đích, thuật toán tăng số nút mở rộng, sinh các trạng thái con hợp lệ và cập nhật những đường đi có chi phí tốt hơn. Như vậy, **đích được kiểm tra khi lấy trạng thái khỏi OPEN, sau khi loại phần tử cũ và trước khi mở rộng**, không phải ngay khi sinh trạng thái con.

Khi $h(s)$ không ước lượng cao hơn chi phí tối ưu còn lại và thuật toán cho phép mở rộng lại trạng thái nếu tìm được đường đi rẻ hơn, lần đầu lấy đích khỏi OPEN sẽ cho chi phí tối ưu. Với $h(s) = 0$ ở mọi trạng thái, A* chọn trạng thái theo chi phí tích lũy như UCS.

3.4.2 Hàm chi phí tích lũy ($g(s)$)

Trạng thái xuất phát có $g(s_0) = 0$. Khi xét hành động a từ s đến s', A* tính:

$$g_{\text{mới}} = g(s) + c(s, a, s').$$

Hàm c là `step_cost` dùng chung với UCS, được định nghĩa ở mục 2.5. Chi phí nhỏ nhất của một hành động hợp lệ là $c_{\min} = 1$. Nếu đường đi mới có chi phí thấp hơn giá trị trong `best_g`, thuật toán cập nhật chi phí, trạng thái cha và đưa phần tử mới vào OPEN với độ ưu tiên $g_{\text{mới}} + h(s')$.

3.4.3 Thiết kế và giải thích hàm heuristic ($h(s)$)

Nhóm thiết kế hai hàm heuristic (đáp ứng yêu cầu báo cáo ≥ 2 hàm), trong đó chỉ hàm thứ nhất được gắn vào bộ giải A*:

Bảng 5: Hai heuristic trong đồ án

Ký hiệu	**Tên triển khai**	**Vai trò**
h_1	Manhattan một nửa (có teleport)	Heuristic mặc định của A*
h_2	Manhattan một nửa + phạt tư thế	So sánh trong báo cáo; không dùng trong A*

Tại đích (khối đứng trên lỗ), $h_1 = h_2 = 0$. Mọi trạng thái khác có $h_1 \geq 0$.

Khoảng cách dựa trên tâm khối

Với hai tọa độ biểu diễn khối, tâm được xác định bởi:

$$\bar{r} = \frac{r_1 + r_2}{2}, \qquad \bar{c} = \frac{c_1 + c_2}{2}.$$

Khoảng cách Manhattan từ tâm đến ô $t = (r_t, c_t)$ là:

$$d_{\text{Man}}(s, t) = |\bar{r} - r_t| + |\bar{c} - c_t|.$$

Một hành động di chuyển không qua ô dịch chuyển làm tâm thay đổi nhiều nhất 1,5 ô. Mã nguồn dùng hệ số chia 2 để lấy một cận dưới đơn giản cho số hành động cần thiết.

Nếu khối đang tách và hai khối con cách nhau d ô theo khoảng cách Manhattan, cần ít nhất $\max(0, d - 1)$ hành động di chuyển để chúng đứng cạnh nhau và ghép lại. Gọi cận này là $m(s)$; với khối liền, $m(s) = 0$.

Từ đó, cận dưới chi phí để đến một ô cần khối đứng, khi không đi qua ô dịch chuyển, được tính bằng:

$$\text{roll}(s, t) = c_{\min} \max\left(\frac{d_{\text{Man}}(s, t)}{2}, m(s)\right).$$

Hai thành phần lấy giá trị lớn hơn thay vì cộng, vì các bước đưa hai khối con lại gần nhau

cũng góp phần di chuyển tâm. Cộng hai cận có thể tính trùng cùng một phần đường đi.

Xét các ô tách và dịch chuyển khối

Khoảng cách đến đích có thể giảm mạnh khi khối đi qua một ô dịch chuyển. Vì vậy, chỉ dùng $\text{roll}(s, \text{goal})$ có thể ước lượng cao hơn chi phí thật. Hàm h_1 xét cả khả năng đến đích trực tiếp và khả năng đi qua các ô này:

$$h_1(s) = \min \left\{ \text{roll}(s, \text{goal}), \min_j \left[\text{roll}(s, T_j) + H(L_j) \right] \right\}.$$

Trong đó, T_j là ô tách và dịch chuyển thứ j; L_j là trạng thái sau khi kích hoạt ô đó, có xét việc ghép lại nếu hai khối con được đặt cạnh nhau. $H(L_j)$ là cận dưới chi phí còn lại từ L_j đến đích, có tính đến khả năng tiếp tục qua các ô dịch chuyển khác.

Để tính H, mã nguồn lập một đồ thị phụ với mỗi nút là một trạng thái sau dịch chuyển. Cạnh từ L_i đến đích có trọng số $\text{roll}(L_i, \text{goal})$; cạnh từ L_i đến L_j có trọng số $\text{roll}(L_i, T_j)$. Cạnh thứ hai ước lượng đoạn đi đến ô T_j, bao gồm hành động kích hoạt dịch chuyển đến L_j. Các giá trị được cập nhật lặp cho đến khi không còn cải thiện hoặc đạt số vòng lặp bằng số ô dịch chuyển. Kết quả được lưu trong bộ nhớ đệm.

Nếu màn không có ô dịch chuyển, công thức rút gọn thành $h_1(s) = \text{roll}(s, \text{goal})$.

Ví dụ về vai trò của dịch chuyển

Xét một hàng gồm 28 ô, đánh số cột từ 0 đến 27. Khối xuất phát ở cột 6, đích ở cột 24. Ô tách ở cột 3 đưa hai khối con đến cột 20 và 23; ô tách ở cột 9 đưa chúng đến cột 18 và 20. Sau dịch chuyển, khối con ở tọa độ thứ nhất được điều khiển.

Lời giải tối ưu gồm hai hành động trái rồi ba hành động phải, có tổng chi phí 9. Sau hành động trái đầu tiên, cận chỉ dựa trên Manhattan/2 là 9,75, trong khi chi phí còn lại của đường tối ưu chỉ là 7. Ví dụ này cho thấy phải xét dịch chuyển khi xây dựng cận dưới. Nó không phải phản ví dụ về tính consistent của h_1 đã có thành phần dịch chuyển.

Heuristic phụ h_2

$$h_2(s) = h_1(s) + \begin{cases} c_{\min} & \text{nếu khối nằm hoặc đang tách} \\ 0 & \text{nếu khối đứng nguyên} \end{cases}$$

Ý định là nhắc rằng lỗ chỉ nhận khối đứng. Tuy nhiên bước lăn cuối thường vừa dịch tâm vừa đổi tư thế; phần dịch chuyển đã được phản ánh trong h_1. Cộng thêm $c_{\min}$ dễ đếm trùng. Trên cạnh có chi phí $c_{\min}$ (ví dụ bước đứng lên teleport), h_2 có thể lớn hơn chi phí còn lại thực tế, nên không đảm bảo admissible trong trường hợp tổng quát. Bộ giải A* chỉ dùng h_1.

3.4.4 Lập luận về tính admissible và/hoặc consistent đối với hàm chi phí đã chọn

Gọi $h^*(s)$ là chi phí nhỏ nhất thực sự từ s đến đích. Heuristic là **admissible** nếu $h(s) \leq h^*(s)$ với mọi trạng thái. Heuristic là **consistent** nếu với mọi bước hợp lệ từ s đến s':

$$h(s) \leq c(s, a, s') + h(s').$$

Cận dưới của h_1

Trên một đoạn đường không có dịch chuyển, mỗi hành động làm tâm thay đổi không quá 2 ô và có chi phí ít nhất $c_{\min}$. Trước khi khối có thể đứng trên đích hoặc kích hoạt ô tách, hai khối con cũng phải ghép lại. Vì vậy, $\text{roll}(s, t)$ không vượt chi phí của đoạn đường thực tế đến ô t.

Nếu một đường đến đích không dùng dịch chuyển, chi phí của nó không nhỏ hơn $\text{roll}(s, \text{goal})$. Nếu đường đó dùng dịch chuyển, chia nó thành các đoạn tại những lần kích hoạt ô dịch chuyển. Mỗi đoạn có một cạnh tương ứng trong đồ thị phụ với trọng số không lớn hơn chi phí đoạn thực tế. Khoảng cách ngắn nhất trong đồ thị phụ vì vậy không vượt chi phí của phần đường thực tế còn lại.

Hàm h_1 lấy giá trị nhỏ nhất giữa các lựa chọn trên, nên $h_1(s) \leq h^*(s)$. Các trọng số ước

lượng đều không âm, nhưng không nhất thiết lớn hơn hoặc bằng $c_{\min}$; cần phân biệt chúng với chi phí của hành động thực tế. Bỏ qua ô trống, ô dễ vỡ và cấu hình cầu khi ước lượng chỉ làm bài toán bớt ràng buộc, không làm cận dưới tăng vượt chi phí thực. Tại đích, $h_1 = 0$.

Phạm vi xét tính consistent

Khi màn không có dịch chuyển, thành phần khoảng cách chia 2 giảm nhiều nhất 0,75 trong một bước, còn cận số bước ghép khối giảm nhiều nhất 1. Do đó, giá trị lớn hơn của hai thành phần giảm không quá 1. Kết hợp với chi phí bước đi ít nhất $c_{\min} = 1$, ta có bất đẳng thức consistent. Kiểm thử hiện tại cũng duyệt các trạng thái có thể đến được trên màn 01 và 04 để kiểm tra tính chất này.

Với màn có dịch chuyển, báo cáo chưa đưa ra khẳng định consistent cho mọi trường hợp. Cài đặt A* không dựa vào giả thiết đó: thuật toán cho phép mở rộng lại trạng thái khi tìm được đường có chi phí thấp hơn. Tính admissible cùng cơ chế này và việc kiểm tra đích khi lấy nút ra là cơ sở bảo đảm tối ưu.

Các kiểm thử hiện có đối chiếu chi phí lời giải A* với UCS trên 10 màn, đồng thời kiểm tra h_1 không vượt chi phí còn lại dọc theo các lời giải do UCS tìm được. Đây là kiểm chứng trên các trường hợp cụ thể, bổ sung cho lập luận ở trên, không thay thế chứng minh cho mọi trạng thái.

Bảng 6: Tóm tắt hai heuristic

Heuristic	**Dùng trong A***	**Admissible**	**Consistent**
h_1	Có	Có	Có nếu không có teleport; không đảm bảo khi có teleport
h_2	Không	Không (tổng quát)	Không

3.4.5 Cách áp dụng vào Bloxorz

A* sử dụng biểu diễn trạng thái, các hành động và quy tắc chuyển trạng thái ở chương 2. Luật di chuyển và kiểm tra đích được gọi qua các hàm dùng chung; chi phí bước đi được lấy từ `solvers/costs.py`.

Bộ giải khởi tạo khối đứng ở vị trí xuất phát với cấu hình cầu ban đầu. Khi mở rộng một

trạng thái, thuật toán thử `UP`, `DOWN`, `LEFT`, `RIGHT` và thêm `SWITCH_BLOCK` nếu khối đang tách. Các bước không hợp lệ bị loại bỏ.

Mỗi phần tử trong OPEN có dạng $(f, \text{số thứ tự}, g, s)$. Số thứ tự tăng dần giúp chọn phần tử được đưa vào trước khi hai giá trị f bằng nhau, tránh so sánh trực tiếp các trạng thái. Khi gặp đích, bộ giải lần ngược thông tin trạng thái cha để trả về chuỗi hành động; khi OPEN rỗng, bộ giải báo không tìm thấy lời giải. Nếu xuất phát đã là đích, chuỗi hành động rỗng và số nút mở rộng bằng 0.

3.4.6 Nhận diện và xử lý trạng thái lặp

Từ điển `best_g` lưu chi phí thấp nhất đã biết đến mỗi trạng thái. Thuật toán chỉ cập nhật trạng thái cha và thêm phần tử mới vào OPEN khi tìm được g nhỏ hơn; các đường có chi phí bằng hoặc lớn hơn bị bỏ qua.

Do không xóa ngay phần tử cũ khỏi hàng đợi, một trạng thái có thể xuất hiện nhiều lần trong OPEN. Khi lấy ra, phần tử có g lớn hơn `best_g` bị bỏ qua và không làm tăng bộ đếm nút mở rộng.

A* không đánh dấu đóng vĩnh viễn các trạng thái đã mở rộng. Nếu sau đó tìm được đường đi rẻ hơn, trạng thái được đưa lại vào OPEN và có thể được mở rộng tiếp. Vì vậy, số nút báo cáo là số lượt mở rộng, có thể lớn hơn số trạng thái phân biệt. Quy ước đo chi tiết được trình bày ở mục 4.3.

4 Thực nghiệm và đánh giá — 20 điểm

4.1 Tập kiểm thử cố định dùng chung cho các thuật toán

Để so sánh hiệu năng của BFS, DFS, UCS và A*, nhóm sử dụng một tập kiểm thử cố định gồm 10 màn chơi, được lưu dưới dạng JSON trong thư mục `stages/`. Mỗi tệp mô tả cấu trúc bàn chơi, vị trí xuất phát, vị trí đích và cấu hình các thành phần đặc biệt như cầu, công tắc và ô tách khối. Danh sách đầu vào được khai báo cố định trong chương trình thực nghiệm, bảo đảm cả bốn thuật toán đều được đánh giá trên cùng một tập dữ liệu.

Trong mỗi màn chơi, các thuật toán sử dụng chung đầu vào, mô hình trạng thái, các quy tắc chuyển trạng thái và điều kiện đạt mục tiêu. Trạng thái ban đầu được xác định thống nhất từ vị trí xuất phát và cấu hình cầu của bản đồ, với khối liền ở tư thế đứng.

Danh sách màn chơi và mã SHA-256 của từng tệp được lưu trong `results/benchmark_metadata.json` để kiểm tra lại các tệp đã dùng trong thực nghiệm.

Sau mỗi lần tìm được lời giải, chương trình phát lại toàn bộ chuỗi hành động từ trạng thái ban đầu. Lời giải chỉ được chấp nhận khi mọi chuyển trạng thái đều hợp lệ và trạng thái cuối cùng là khối liền đứng trên ô đích. Bước kiểm tra này được thực hiện ngoài phạm vi đo thời gian và bộ nhớ của thuật toán.

4.2 Mô tả đặc điểm và các nhóm kiểm thử

Tập kiểm thử bao gồm các màn có cấu trúc đường đi và cơ chế tương tác khác nhau. Các màn được phân nhóm theo nội dung bản đồ và cấu hình thực tế trong tệp JSON. Số thứ tự chỉ dùng để phân biệt các màn và xác định thứ tự chạy, không thể hiện độ khó tăng dần.

Bảng 7: Các nhóm kiểm thử và mục đích đánh giá

Nhóm kiểm thử	**Màn chơi**	**Đặc điểm chính**	**Mục đích đánh giá**
Sàn thường	01, 03, 06	Không có ô đặc biệt; gồm đường đi cơ bản, lối hẹp và mê cung	Đánh giá khả năng tìm đường khi chủ yếu phải xét vị trí và tư thế khối
Ô dễ vỡ	04	Có các ô cho phép khối liền nằm trên nhưng không được đứng thẳng	Đánh giá khả năng tìm đường khi tư thế khối bị hạn chế
Công tắc và cầu	02, 05, 07	Có công tắc điều khiển cầu; màn 05 kết hợp các thao tác bật, tắt và đảo trạng thái cầu	Đánh giá khả năng tìm đường khi việc đóng/mở cầu làm thay đổi các lối đi
Tách khối	08, 09	Có ô tách và dịch chuyển khối, cho phép điều khiển hai khối con rồi ghép lại	Đánh giá khả năng phối hợp di chuyển hai khối con và đổi khối điều khiển
Tách khối kết hợp công tắc và cầu	10	Kết hợp cơ chế tách khối với công tắc và nhiều nhóm cầu	Đánh giá khả năng xử lý đồng thời nhiều thành phần của trạng thái

Các nhóm kiểm thử cho phép đánh giá việc xử lý vị trí và tư thế khối, điều kiện di chuyển trên ô dễ vỡ, cấu hình cầu và trạng thái của hai khối con. Khi cầu đóng hoặc mở, các hành động hợp lệ có thể thay đổi dù khối vẫn ở cùng vị trí và tư thế. Vì vậy, việc nhận diện trạng thái lặp phải xét cả những thông tin này. Việc phân nhóm dựa trên nội dung và cấu hình trong tệp JSON, không dựa trên tên tệp.

4.3 Phương pháp đo các chỉ số

Nhóm sử dụng bốn chỉ số để đánh giá các thuật toán: thời gian tìm kiếm, mức sử dụng bộ nhớ đỉnh, số nút mở rộng và độ dài lời giải. Thực nghiệm được thực hiện lần lượt từ màn 01 đến màn 10; với mỗi màn, các thuật toán chạy theo thứ tự BFS, DFS, UCS và A*. Mỗi thuật toán được chạy 5 lần liên tiếp trên một màn, mỗi lần khởi tạo một bộ giải mới và không chạy thử trước khi đo. Theo cách tổ chức này, có tổng cộng 200 lượt chạy dự kiến; số lượt thành công được ghi nhận riêng cho từng thuật toán trên từng màn.

Bên cạnh bốn chỉ số chính, nhóm ghi nhận thêm số lần lăn để phân biệt các bước di chuyển với thao tác đổi khối điều khiển. Cách tính được trình bày cùng độ dài lời giải ở mục 4.3.4. Phạm vi đo bao gồm khởi tạo các cấu trúc phục vụ tìm kiếm, thực hiện tìm kiếm và dựng lại chuỗi hành động khi tìm được đích. Các công việc nạp bản đồ, khởi tạo đối tượng bộ giải, phát lại để kiểm tra lời giải, tính chi phí đường đi, ghi kết quả và tạo biểu đồ được thực hiện ngoài phạm vi này.

Riêng A*, hàm heuristic mặc định tính trước một số giá trị để ước lượng chi phí của các đường đi qua ô tách khối. Trước mỗi lượt A*, chương trình thực nghiệm xóa bộ nhớ đệm heuristic rồi mới bắt đầu đo. Vì vậy, mỗi lượt đều bắt đầu với bộ nhớ đệm rỗng, và thời gian tính toán cùng bộ nhớ cấp phát cho các giá trị này được tính trong phạm vi đo. Trong cùng một lượt, A* vẫn dùng lại các giá trị đã tính để tránh tính toán lặp.

Thời gian và mức sử dụng bộ nhớ đỉnh được tổng hợp bằng trung vị của các lượt chạy có lời giải đã được kiểm tra và chấp nhận. Cách tổng hợp này giúp hạn chế ảnh hưởng của những dao động bất thường giữa các lượt. Nếu chỉ có một số lượt thành công, chương trình lấy trung vị của các lượt đó và ghi rõ số lượt thành công trong tổng số 5 lượt. Khi các lượt có trạng thái khác nhau, kết quả tổng hợp được đánh dấu là thất bại. Nếu không có lượt thành công, giá trị trung vị được để trống, không gán bằng 0.

Các trường hợp không tìm thấy lời giải, gặp lỗi khi chạy hoặc trả về lời giải không hợp lệ được ghi lại bằng các trạng thái riêng trong dữ liệu chi tiết. Chương trình chưa có cơ chế giới hạn thời gian chạy. Số nút mở rộng và độ dài lời giải được đối chiếu giữa các lần lặp. Nếu các giá trị khác nhau, ô dữ liệu tương ứng được để trống và kết quả được đánh dấu không nhất quán, thay vì lấy trung bình.

4.3.1 Thời gian tìm kiếm

Thời gian tìm kiếm được đo bằng hàm `time.perf_counter()` của Python. Đây là bộ đếm thời gian có độ phân giải cao, phù hợp để đo khoảng thời gian thực thi của thuật toán.

Gọi t_{start} là thời điểm bắt đầu đo và t_{end} là thời điểm hoàn thành tìm kiếm cùng bước dựng lại đường đi. Thời gian của một lần chạy được tính như sau:

$$T = t_{\text{end}} - t_{\text{start}}.$$

Đơn vị đo là giây. Thời gian được ghi nhận bao gồm cả ảnh hưởng của việc theo dõi bộ nhớ, do hai phép đo được thực hiện đồng thời.

4.3.2 Mức sử dụng bộ nhớ đỉnh

Mức sử dụng bộ nhớ đỉnh được đo bằng thư viện `tracemalloc`. Nếu thư viện chưa theo dõi bộ nhớ, chương trình bật theo dõi trước khi tìm kiếm và dừng sau khi lấy số đo, kể cả khi xảy ra lỗi. Nếu việc theo dõi đã được bật từ trước, chương trình dùng `reset_peak()` để bắt đầu ghi nhận lại mức đỉnh và không tắt theo dõi sau phép đo. Thao tác này không loại bỏ lượng bộ nhớ đang được theo dõi tại thời điểm đặt lại.

Sau khi hoàn thành tìm kiếm và dựng lại đường đi, chương trình lấy lượng bộ nhớ lớn nhất đã ghi nhận bằng `tracemalloc.get_traced_memory()`. Khi chạy thực nghiệm theo chương trình hiện tại, việc theo dõi được bật và tắt riêng cho từng lượt. Bộ nhớ đệm heuristic của A* được xóa trước khi bật theo dõi, nên phần bộ nhớ cấp phát để tính và lưu lại dữ liệu này được ghi nhận trong từng lượt.

Gọi P_{bytes} là lượng bộ nhớ đỉnh được ghi nhận theo byte. Giá trị báo cáo được chuyển sang MiB theo công thức:

$$M_{\text{peak}} = \frac{P_{\text{bytes}}}{1024^2}.$$

Chỉ số này cho biết lượng bộ nhớ lớn nhất mà `tracemalloc` ghi nhận từ các cấp phát của Python trong phạm vi đo. Phần bộ nhớ đó bao gồm các cấu trúc như tập trạng thái đã thăm, hàng đợi hoặc ngăn xếp, hàng đợi ưu tiên và bảng lưu trạng thái cha để dựng lại đường đi. Chỉ số không đại diện cho toàn bộ RAM của tiến trình hay mức sử dụng bộ nhớ của hệ điều hành.

4.3.3 Số nút mở rộng

Một nút được tính là đã mở rộng khi thuật toán lấy trạng thái tương ứng ra khỏi hàng đợi, ngăn xếp hoặc hàng đợi ưu tiên và bắt đầu xét các hành động để sinh trạng thái kế tiếp. Bộ đếm tăng một lần cho mỗi lượt mở rộng, không tăng riêng cho từng hành động được thử hoặc từng trạng thái con được sinh ra.

Đối với BFS và DFS, bộ đếm tăng sau khi lấy một trạng thái ra khỏi hàng đợi hoặc ngăn xếp. Hai thuật toán kiểm tra đích khi sinh trạng thái kế tiếp, nên trạng thái đích được phát hiện sẽ không được mở rộng.

Đối với UCS và A*, sau khi lấy một phần tử khỏi hàng đợi ưu tiên, thuật toán so sánh chi phí của phần tử đó với chi phí tốt nhất đã lưu cho trạng thái tương ứng. Nếu chi phí lớn hơn, phần tử bị bỏ qua và không được tính vào số nút mở rộng. Nếu trạng thái là đích, thuật toán kết thúc trước khi tăng bộ đếm; nếu chưa phải đích, thuật toán tăng bộ đếm rồi xét các trạng thái kế tiếp.

Do đó, chỉ số được báo cáo là **số lượt mở rộng**, không nhất thiết bằng số trạng thái phân biệt đã được mở rộng. Nếu một trạng thái được mở rộng lại sau khi tìm thấy đường đi có chi phí tốt hơn, lượt mở rộng mới vẫn được tính. Trạng thái đích không được tính vào chỉ số này; nếu trạng thái ban đầu đã là đích, cả bốn thuật toán đều trả về số nút mở rộng bằng 0.

4.3.4 Độ dài lời giải — số hành động

Độ dài lời giải được xác định bằng số phần tử trong chuỗi hành động mà thuật toán trả về. Với lời giải:

$$\pi = (a_1, a_2, \ldots, a_k),$$

độ dài được tính là:

$$L(\pi) = k.$$

Mỗi hành động `UP`, `DOWN`, `LEFT`, `RIGHT` hoặc `SWITCH_BLOCK` đều được tính là một đơn vị. Như vậy, thao tác đổi khối đang điều khiển khi khối bị tách vẫn được tính vào độ dài, dù thao tác này không làm thay đổi vị trí hai khối con.

Quy ước trên phân biệt độ dài lời giải với số lần di chuyển theo bốn hướng. Chẳng hạn, một lời giải gồm 10 hành động di chuyển và 2 thao tác đổi khối có độ dài 12 hành động. Các hiệu ứng tự động của một hành động, như kích hoạt công tắc, dịch chuyển qua ô tách hoặc ghép hai khối, không được tính thêm thành hành động riêng.

Số lần lăn chỉ đếm các hành động `UP`, `DOWN`, `LEFT` và `RIGHT`, không tính `SWITCH_BLOCK`. Mỗi lần di chuyển khối liền hoặc khối con theo một trong bốn hướng đều được tính một lần. Chỉ số này được tính khi phát lại lời giải để kiểm tra tính hợp lệ, ngoài phạm vi đo thời gian và bộ nhớ tìm kiếm. Vì vậy, lời giải trong ví dụ trên có 12 hành động nhưng chỉ có 10 lần lăn. Số lần lăn cũng phải giống nhau giữa các lượt chạy thành công; chương trình không lấy trung bình khi các giá trị khác nhau.

BFS bảo đảm ít hành động nhất khi tính cả `SWITCH_BLOCK`, không bảo đảm riêng số lần lăn ít nhất. Việc bổ sung chỉ số này không thay đổi cách tìm kiếm hay cách tính độ dài lời giải của các bộ giải.

Độ dài lời giải cũng được phân biệt với tổng chi phí đường đi:

$$C(\pi) = \sum_{i=1}^{k} c(s_{i-1}, a_i, s_i).$$

Do chi phí của các hành động không đồng nhất, hai lời giải có cùng số hành động có thể có tổng chi phí khác nhau. BFS tìm lời giải có ít hành động nhất, còn UCS tìm lời giải có tổng chi phí nhỏ nhất. A* cũng bảo đảm tổng chi phí nhỏ nhất khi heuristic không ước lượng cao hơn chi phí tối ưu còn lại và thuật toán cho phép xét lại trạng thái khi tìm được đường đi có chi phí thấp hơn. DFS không bảo đảm tìm được lời giải tối ưu. Vì vậy, khi so sánh độ dài lời giải, cần xét đến mục tiêu tối ưu của từng thuật toán.

Chương trình kiểm tra độ dài được báo cáo có bằng số hành động thực tế hay không trước khi chấp nhận kết quả. Trường hợp không tìm thấy lời giải được ghi nhận bằng trạng thái riêng, không được xem là một lời giải hợp lệ có độ dài bằng 0.

4.4 Bảng tổng hợp kết quả thực nghiệm

Hai bảng dưới đây trình bày kết quả của BFS, DFS, UCS và A* trên 10 màn chơi theo bốn chỉ số đã nêu ở mục 4.3.

Bảng 8: Kết quả thực nghiệm trên các màn 01–05

Màn chơi	**Thuật toán**	**Thời gian (giây)**	**Bộ nhớ đỉnh (MiB)**	**Số nút mở rộng**	**Độ dài lời giải (hành động)**
01	BFS	0.001205	0.01344	48	7
01	DFS	0.001714	0.02121	63	37
01	UCS	0.001855	0.01405	64	7
01	A*	0.001024	0.01148	34	7
02	BFS	0.007690	0.05099	263	17
02	DFS	0.009863	0.09796	315	64
02	UCS	0.009635	0.05498	283	17
02	A*	0.008175	0.04869	247	17
03	BFS	0.001887	0.02580	81	19
03	DFS	0.001082	0.01162	41	32
03	UCS	0.002416	0.02113	87	19
03	A*	0.002306	0.02109	85	19
04	BFS	0.001506	0.01390	68	28
04	DFS	0.002386	0.02473	98	28
04	UCS	0.001852	0.01323	69	28
04	A*	0.001722	0.01293	68	28
05	BFS	0.008015	0.08453	308	33
05	DFS	0.006893	0.05290	240	175
05	UCS	0.009653	0.05595	316	33
05	A*	0.008193	0.05202	283	33

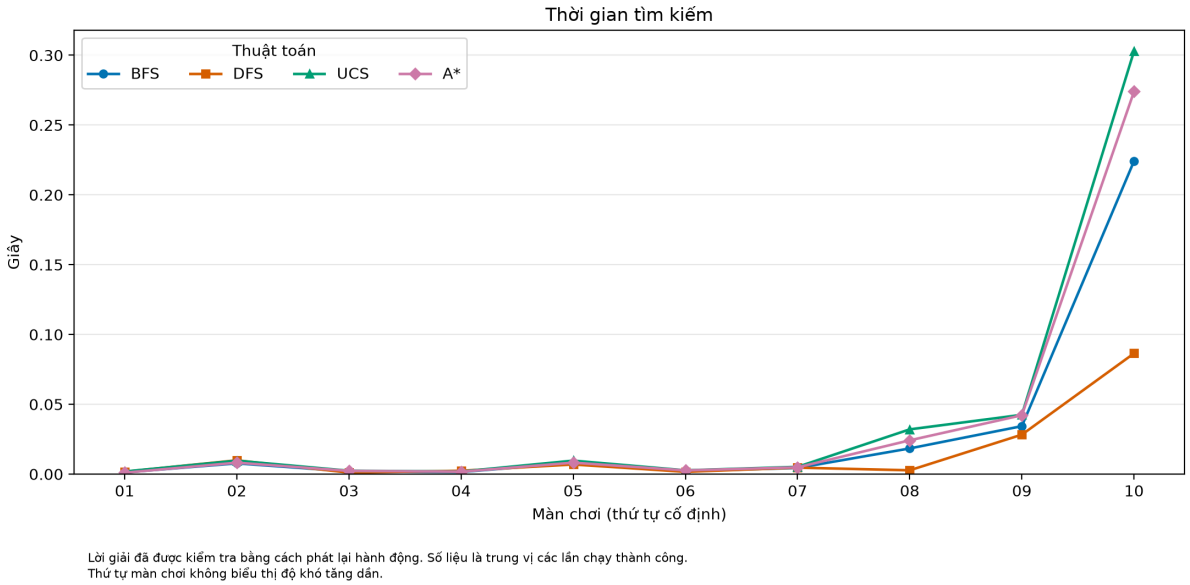
Bảng 9: Kết quả thực nghiệm trên các màn 06–10

Màn chơi	**Thuật toán**	**Thời gian (giây)**	**Bộ nhớ đỉnh (MiB)**	**Số nút mở rộng**	**Độ dài lời giải (hành động)**
06	BFS	0.002134	0.02642	102	35
06	DFS	0.001627	0.02121	66	54
06	UCS	0.002713	0.02237	106	35
06	A*	0.002637	0.02239	106	35
07	BFS	0.004381	0.04224	173	44
07	DFS	0.004621	0.04125	169	61
07	UCS	0.005203	0.04200	177	44
07	A*	0.004840	0.04193	176	44
08	BFS	0.018388	0.16534	614	11
08	DFS	0.002717	0.03022	97	44
08	UCS	0.032036	0.19751	884	11
08	A*	0.024196	0.17661	605	11
09	BFS	0.034337	0.33461	1232	25
09	DFS	0.028257	0.39185	916	468
09	UCS	0.042479	0.33522	1331	25
09	A*	0.042144	0.22433	1158	25
10	BFS	0.224189	1.84480	5622	58
10	DFS	0.086471	0.86700	2264	1314
10	UCS	0.302919	1.90676	6332	58
10	A*	0.273997	1.75974	5423	58

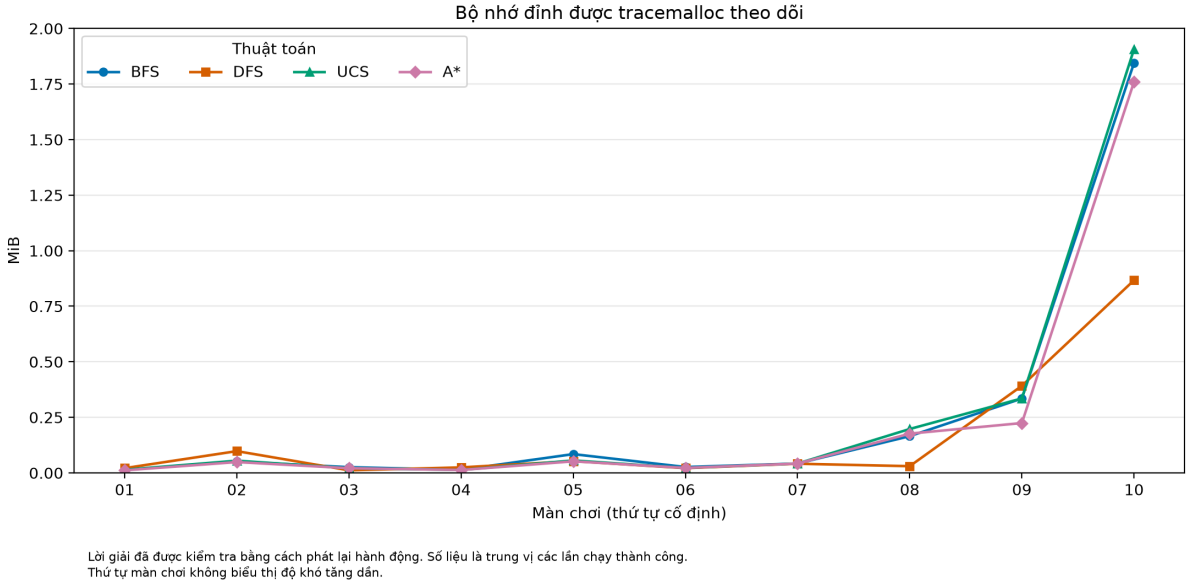
Ghi chú. Tất cả 40 cặp màn chơi–thuật toán đều thành công trong 5/5 lượt chạy. Thời gian và bộ nhớ là trung vị của 5 lượt, lần lượt được làm tròn đến 6 và 5 chữ số sau dấu thập phân. Độ dài lời giải tính cả thao tác đổi khối điều khiển (`SWITCH_BLOCK`). Kết quả đầy đủ và chi tiết hơn được lưu trong `results/benchmark_summary.csv`.

4.5 Biểu đồ so sánh các thuật toán

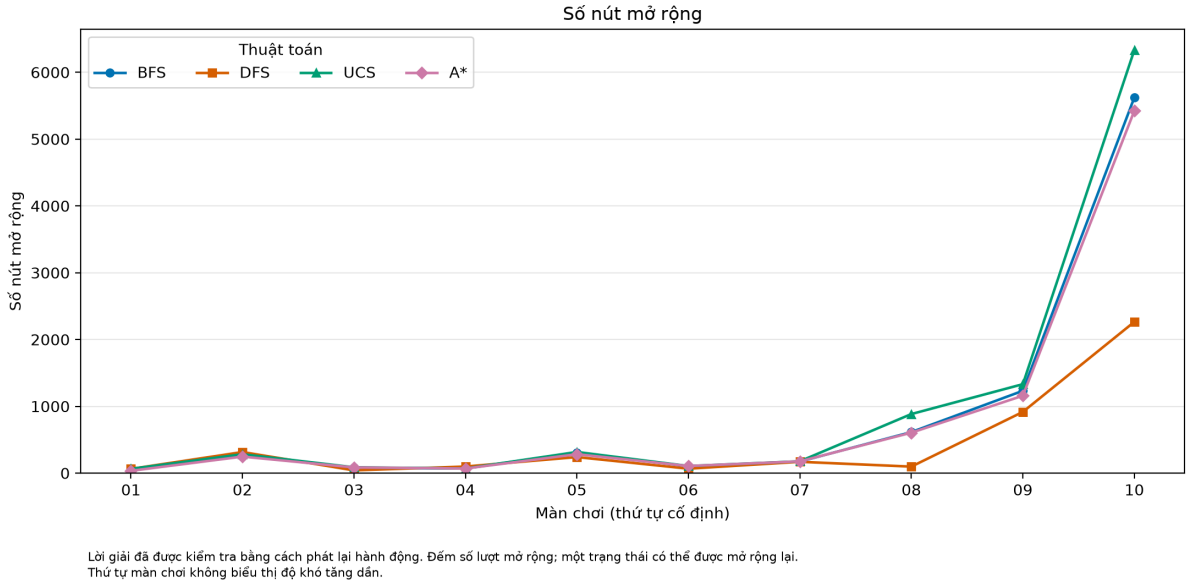
Năm biểu đồ dưới đây so sánh BFS, DFS, UCS và A* trên 10 màn chơi, dựa trên cùng số liệu ở mục 4.4. Bên cạnh bốn chỉ số chính, biểu đồ số lần lăn cho biết số bước di chuyển khi không tính thao tác đổi khối điều khiển. Thời gian và bộ nhớ là trung vị của 5 lượt chạy thành công. Trục ngang thể hiện số thứ tự màn chơi, không thể hiện độ khó tăng dần.



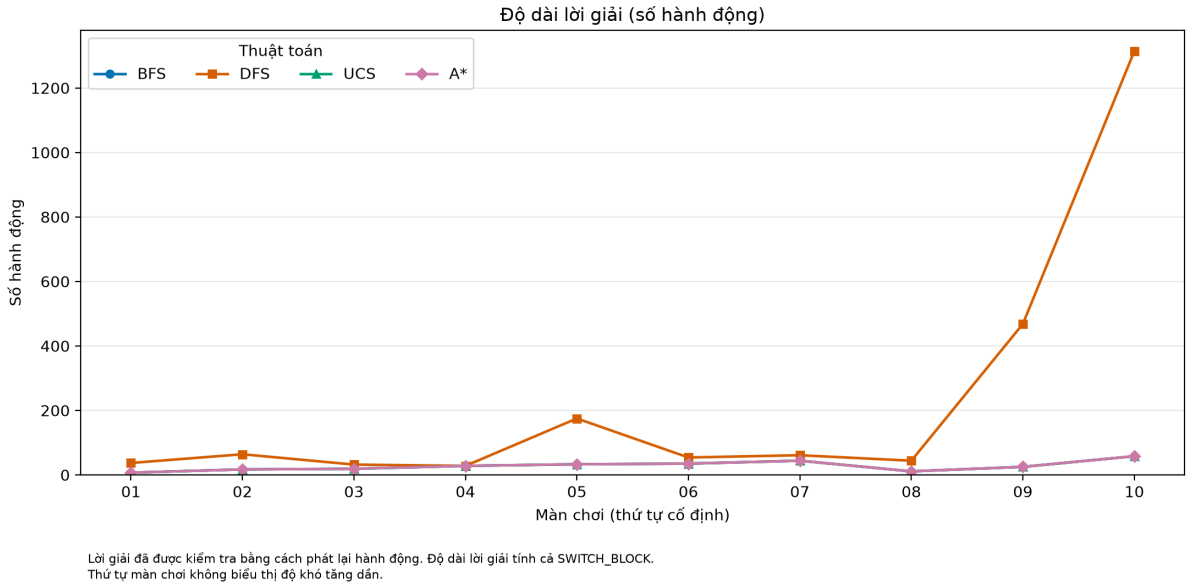
Hình 1: Thời gian tìm kiếm của các thuật toán, tính bằng giây.



Hình 2: Mức sử dụng bộ nhớ đỉnh được tracemalloc theo dõi, tính bằng MiB.

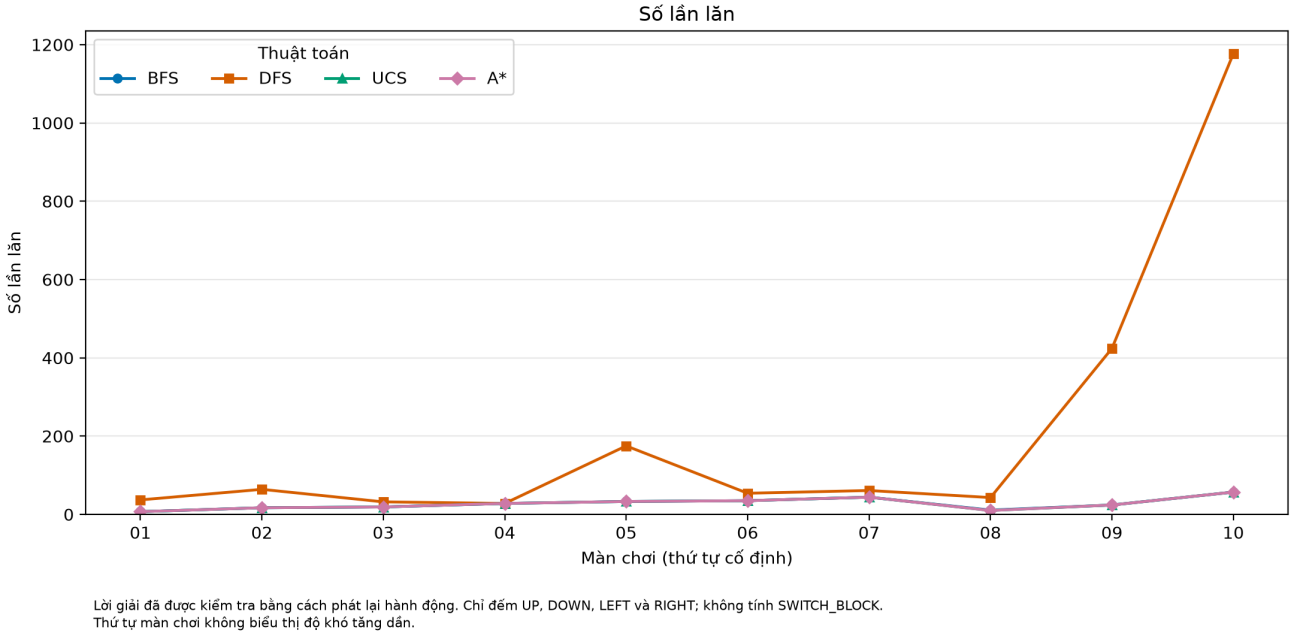


Hình 3: Số lượt mở rộng nút của các thuật toán. Một trạng thái có thể được mở rộng nhiều lần.



Hình 4: Độ dài lời giải của các thuật toán, tính cả thao tác đổi khối điều khiển (SWITCH_BLOCK).

Ghi chú: BFS, UCS và A* cho lời giải có cùng số hành động trên cả 10 màn nên ba đường biểu diễn chồng lên nhau. Đường A* được vẽ sau nên che hai đường còn lại; biểu đồ vẫn có đủ dữ liệu của bốn thuật toán.



Hình 5: Số lần lăn của các thuật toán, chỉ tính UP, DOWN, LEFT và RIGHT, không tính SWITCH_BLOCK.

Ghi chú: UCS và A* có cùng số lần lăn trên cả 10 màn nên hai đường chồng lên nhau. BFS cũng trùng với hai thuật toán này ở 9 màn; riêng màn 08, BFS có 11 lần lăn, còn UCS và A* có 10 lần.

4.6 Phân tích kết quả và ưu, nhược điểm của từng thuật toán

Cả bốn thuật toán đều tìm được lời giải hợp lệ trên 10 màn chơi, với 5/5 lượt chạy thành công ở mỗi màn. Tuy nhiên, thời gian tìm kiếm, bộ nhớ và độ dài lời giải có sự khác biệt rõ rệt. Tìm được lời giải nhanh không đồng nghĩa với tìm được lời giải có ít hành động nhất.

BFS

BFS phù hợp khi mục tiêu là tìm lời giải có ít hành động nhất. Trong bộ kiểm thử này, độ dài lời giải của BFS bằng UCS và A* trên cả 10 màn. BFS cũng chạy nhanh hơn UCS và A* ở 9/10 màn, cho thấy cách duyệt theo từng mức vẫn hiệu quả trên các bản đồ đang xét. Hạn chế của BFS là phải lưu nhiều trạng thái chờ xét và trạng thái đã thăm. Ở màn 10, thuật toán mở rộng 5.622 nút, mất khoảng 0,224 giây và dùng bộ nhớ đỉnh khoảng 1,845 MiB. So với các màn còn lại, cả ba chỉ số đều tăng rõ rệt. Khi có nhiều trạng thái cần xét, yêu cầu về thời gian và bộ nhớ của BFS cũng trở nên đáng chú ý hơn.

DFS

DFS tìm được lời giải nhanh nhất ở 6/10 màn, nhưng thường trả về chuỗi hành động dài hơn các thuật toán còn lại. Chẳng hạn, ở màn 10, DFS chỉ mất khoảng 0,086 giây và mở rộng 2.264 nút, trong khi BFS cần khoảng 0,224 giây và 5.622 nút. Đổi lại, lời giải của DFS dài tới 1.314 hành động, so với 58 hành động của BFS, UCS và A*.

Kết quả này thể hiện ảnh hưởng của cách đi sâu theo một nhánh: DFS có thể sớm gặp đích, nhưng cũng có thể đi vòng qua nhiều trạng thái trước khi đến nơi. Hiệu quả phụ thuộc vào thứ tự xét hành động và cấu trúc bản đồ. DFS cũng không luôn dùng ít bộ nhớ hơn: ở màn 09, thuật toán mở rộng ít nút hơn BFS nhưng dùng khoảng 0,392 MiB, cao hơn mức 0,335 MiB của BFS. Vì vậy, không thể suy ra mức sử dụng bộ nhớ chỉ từ số nút mở rộng; còn phải xét các trạng thái đang chờ, tập đã thăm và thông tin lưu để dựng lại đường đi.

UCS

UCS chọn trạng thái theo tổng chi phí từ điểm xuất phát và bảo đảm tìm lời giải có chi phí nhỏ nhất khi chi phí các hành động dương. Mục tiêu này khác với việc giảm số hành động, dù trong bộ kiểm thử hiện tại UCS và BFS cho lời giải có cùng độ dài trên cả 10 màn.
Về hiệu năng, UCS mở rộng nhiều nút hơn BFS ở cả 10 màn và có thời gian tìm kiếm cao nhất trong bốn thuật toán ở 8/10 màn. Ở màn 10, UCS mở rộng 6.332 nút, mất khoảng 0,303 giây và dùng khoảng 1,907 MiB bộ nhớ đỉnh. UCS phải quản lý hàng đợi ưu tiên và cập nhật chi phí tốt nhất cho từng trạng thái; thứ tự duyệt theo chi phí cũng có thể khiến thuật toán xét thêm trạng thái trước khi đến đích. Do đó, UCS phù hợp khi cần tối ưu chi phí, nhưng không nhất thiết có lợi thế về thời gian nếu chỉ quan tâm đến số hành động.

A*

A* sử dụng heuristic để hướng việc tìm kiếm về phía đích. So với UCS, A* mở rộng ít nút hơn ở 9/10 màn và bằng nhau ở màn 06. Thời gian tìm kiếm của A* cũng thấp hơn UCS trên cả 10 màn. Ví dụ, ở màn 08, số nút mở rộng giảm từ 884 xuống 605, còn thời gian giảm từ khoảng 0,032 giây xuống 0,024 giây. Những kết quả này cho thấy heuristic đã giúp giảm lượng công việc tìm kiếm so với chỉ xét chi phí từ điểm xuất phát.
Tuy vậy, ít nút hơn không có nghĩa là luôn chạy nhanh hơn BFS. Ở màn 10, A* mở rộng 5.423 nút, ít hơn BFS với 5.622 nút, nhưng mất khoảng 0,274 giây so với 0,224 giây của BFS. Việc tính heuristic và quản lý hàng đợi ưu tiên cũng tốn thời gian; phép đo hiện tại còn bao gồm bước tính dữ liệu heuristic sau khi xóa bộ nhớ đệm trước mỗi lượt. Mức cải thiện vì vậy phụ thuộc vào việc heuristic giúp tránh được bao nhiêu lượt mở rộng so với phần tính toán bổ sung. A* hướng đến lời giải có chi phí nhỏ nhất khi đáp ứng các điều kiện bảo đảm tối ưu đã trình bày ở phần thuật toán.

Ảnh hưởng của màn chơi và lựa chọn thuật toán

Số lần lăn cho thấy cùng độ dài lời giải chưa chắc có cùng số bước di chuyển. Ở màn 08, BFS, UCS và A* đều trả về 11 hành động. Tuy nhiên, BFS thực hiện 11 lần lăn, còn UCS và A* thực hiện 10 lần lăn và một thao tác đổi khối điều khiển. Kết quả này không mâu thuẫn với tính tối ưu của BFS, vì BFS giảm tổng số hành động, không tối ưu riêng số lần lăn. Ở màn 10, lời giải DFS gồm 1.314 hành động, trong đó có 1.177 lần lăn; BFS, UCS và A* đều có 58 hành động với 57 lần lăn.

Màn 10 có thời gian, bộ nhớ đỉnh và số nút mở rộng cao nhất đối với cả bốn thuật toán. Màn này kết hợp tách khối, công tắc và cầu, nên quá trình tìm kiếm phải phân biệt nhiều trạng thái hơn ngoài vị trí và tư thế khối. Tuy nhiên, không thể chỉ dựa vào số cơ chế đặc biệt để xếp độ khó: hình dạng đường đi, vị trí đích và thứ tự duyệt cũng ảnh hưởng đến kết quả. Số thứ tự màn vì vậy không thể hiện độ khó tăng dần.

Trong phạm vi 10 màn đã thử nghiệm, BFS là lựa chọn phù hợp khi cần ít hành động; DFS có thể hữu ích khi ưu tiên tìm nhanh một lời giải hợp lệ và chấp nhận đường đi dài. UCS và A* phù hợp với mục tiêu tối ưu chi phí, trong đó A* cho kết quả thời gian tốt hơn UCS ở bộ kiểm thử này. Các nhận xét về thời gian dựa trên trung vị của 5 lượt chạy; những chênh lệch nhỏ chưa đủ để khẳng định một thuật toán luôn tốt hơn trên các bản đồ khác.

5 Tài liệu tham khảo

1. Coolmath Games. *Bloxorz Original Game*. Trò chơi Bloxorz gốc.

2. tkoz0. *bloxorz-solver*. Nguồn bộ dữ liệu bản đồ và lời giải đối chiếu (ground truth).

3. dpquoc. *bloxorz*. Tham khảo cơ chế tách khối (split block).

4. chaitanya100100. *Bloxorz*. Tham khảo thiết kế giao diện 3D, camera và hoạt ảnh.

A Khai báo sử dụng công cụ AI

Nhóm sử dụng AI để hỗ trợ thiết kế, lập trình, review code, kiểm thử và biên soạn báo cáo. Phạm vi sử dụng của từng thành viên được tổng hợp dưới đây; tên công cụ và model được ghi theo khai báo của thành viên.

A.1 Công cụ và phạm vi sử dụng

Thành viên	**Công cụ / Model**	**Phạm vi AI hỗ trợ**
TV1 — Phạm Trường Quỳnh (24120426)	Antigravity IDE: Gemini 3.8 Flash, Claude Opus 4.6; Gemini Pro trên web	Đề xuất kiến trúc; thiết kế `State`; hỗ trợ cài đặt engine, BFS và unit test; hỗ trợ mô hình hóa bài toán và định dạng công thức toán học.
TV2 — Nguyễn Tấn Vinh (24120492)	Claude trên web: Claude Sonnet 5.5; Gemini trên web: Gemini Pro 3.1	Soạn bản nháp DFS, UCS, hàm chi phí và test; hỗ trợ sửa lỗi đo lường; soạn và rà soát phần báo cáo tương ứng.
TV3 — Võ Gia Phúc (24120413)	Cursor IDE: Claude Sonnet 5.5 High / Composer 2.5 Fast	Hỗ trợ cài đặt A*, thiết kế heuristic có teleport, xây dựng test và module đo hiệu năng; hỗ trợ lập luận lý thuyết và biên soạn báo cáo.
TV4 — Thân Nguyễn Quang Triệu (24120475)	Copilot SDK trong VS Code; không xác định tên model cụ thể	Phân tích và tích hợp GUI WebGL với Python API; hỗ trợ sửa tọa độ, orientation, animation, manual play, replay và các lỗi giao diện.
TV5 — Nguyễn Việt Thái (24120441)	ChatGPT: GPT-6 Luna (review code), GPT-6.1 Sol (hỗ trợ viết code)	Review code; hỗ trợ viết `benchmark.py`, `report_data.py` để tạo biểu đồ từ số liệu benchmark và xây dựng phần nhận xét, phân tích thuật toán đã đưa vào báo cáo.

A.2 Prompt tiêu biểu

Thành viên	**Nội dung prompt**	**Đầu ra liên quan**
TV1	Đề xuất kiến trúc tách biệt engine, solver và GUI; thiết kế trạng thái bất biến, chuẩn hóa tọa độ để lưu trong tập visited.	Cấu trúc dự án, `domain/state.py`.
TV1	Hỗ trợ cài đặt chuyển trạng thái, công tắc, tách/ghép khối và BFS; xây dựng test cho các tình huống biên.	`engine/`, `solvers/bfs.py`, bộ test.
TV2	Viết DFS và UCS theo interface có sẵn; tách hàm chi phí dùng chung và kiểm tra quy tắc ô dễ vỡ khi khối đã tách.	`dfs.py`, `ucs.py`, `costs.py`.
TV2	Rà soát phép đo, bổ sung test và phân tích kết quả DFS/UCS từ dữ liệu benchmark.	Test, mã nguồn chỉnh sửa và bản thảo báo cáo.
TV3	Cài A* với `best_g`, kiểm tra đích khi pop và mở lại nút; thiết kế heuristic xét teleport và kiểm tra với UCS.	`astar.py`, `heuristics.py`, test A*.
TV3	Hỗ trợ đo thời gian, bộ nhớ đỉnh và số nút mở rộng; trình bày tính admissible và giới hạn về consistency của heuristic.	`metrics/measurement.py`, nội dung lý thuyết và phương pháp đo.
TV4	Kết nối GUI với API `/stage`, `/solve`, `/transition`; đồng bộ tọa độ và orientation với Python engine.	Frontend WebGL và lớp kết nối API.
TV4	Hỗ trợ replay đường đi có `SWITCH_BLOCK`, hiển thị khối tách/ghép và sửa lỗi chuyển level, camera, life.	Manual play, animation và solver replay.
TV5	Review interface và phép đo; hỗ trợ viết benchmark cho bốn thuật toán trên 10 màn, lặp 5 lần và tổng hợp trung vị theo quy ước thực nghiệm.	`benchmark.py`.
TV5	Viết mã tạo biểu đồ từ số liệu benchmark; đối chiếu biểu đồ với dữ liệu và phân tích thời gian, bộ nhớ, số nút, độ dài lời giải của các thuật toán.	`report_data.py` và phần nhận xét, phân tích thuật toán đã đưa vào báo cáo.

A.3 Kiểm chứng và trách nhiệm

- **TV1:** Đọc hiểu, chỉnh sửa và tích hợp engine/BFS; kiểm chứng luật chơi bằng unit test và chơi thử.
- **TV2:** Kiểm tra DFS/UCS và hàm chi phí bằng test, đối chiếu kết quả với benchmark của nhóm.
- **TV3:** Chạy test, so sánh A* với UCS, đối chiếu bảng với CSV và quyết định thiết kế heuristic cuối cùng.
- **TV4:** Kiểm tra API, manual play, orientation, advanced tiles và replay trên trình duyệt; đối chiếu state với Python engine.
- **TV5:** Chịu trách nhiệm rà soát `benchmark.py`, `report_data.py`, đối chiếu biểu đồ với số liệu benchmark và bảo đảm phần nhận xét, phân tích thuật toán trong báo cáo phù hợp với kết quả thực nghiệm.

Các thành viên chịu trách nhiệm đọc hiểu, đánh giá, chỉnh sửa và tích hợp đầu ra AI trong phần việc của mình. Số liệu báo cáo phải lấy từ kết quả chạy chương trình; các kết luận thực nghiệm cần giới hạn trong phạm vi dữ liệu được kiểm tra. Nhóm chịu trách nhiệm về mã nguồn, số liệu và nội dung báo cáo cuối cùng.